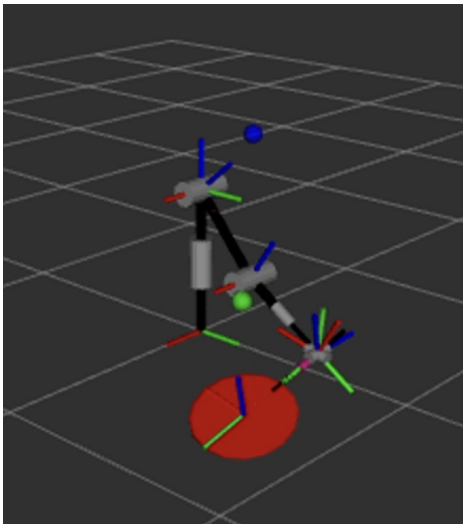
Me 133a final report

Riti Agarwal, Jihyun Jeon

December 10, 2023

1 Introduction



The goal of our project is to create a tennis playing robotic arm. The arm has 6 degrees of freedom with the tip being the paddle. The arm is capable of hitting a ball to a goal point. The ball (the green marker in the image) is spawned with a random trajectory (i.e. random initial position and velocity) such that it heads toward an area around the arm. The goal point (the blue marker in the image) is also randomly generated. Through inverse kinematics, the arm orients itself such that it hits the ball towards the goal point with reasonable joint configuration.

Whenever a ball is launched, the arm is given the trajectory of the ball and the location of the goal. In terms of physics, elastic collision was implemented to be used between the ball and the paddle and the ball and the surroundings (i.e. ground and invisible walls, used to contain the ball when the player misses). While gravity wasn't used in the final demonstration, it is possible to add it as capability of dealing with acceleration in ball trajectory was implemented.

2 Description of Robot

The robot has 6 degree of freedoms. θ_1 connects the robot to the world and allows rotation about the z axis. θ_2 acts as the shoulder, allowing rotation about the x axis, similar to the shoulder's ability to raise or lower the arm. θ_3 is the elbow, allowing rotation around the x axis, contributing to the ability to pitch or tilt the forearm further. θ_4, θ_5 and θ_6 make up the wrist. θ_4 allows rotation about the y axis, similar to the ability of the forearm to rotate about its own axis. θ_5 allows rotation around the z axis, similar to the wrist's ability to rotate in the horizontal plane, and θ_6 allows rotation around the x axis, contributing to the hand's ability to pitch or tilt, kind of how you move your wrist up and down. The z axis of the paddle is normal to the surface and the y axis of the paddle is along the wrist, as can be seen in the image in section 1.

Since the robot is attached to the world frame and cannot "run", the robot can reach any ball within 0.9 meters, or the length of the arm. The base is fixed and there are no redundancies. The tip is a flat cylinder and acts as the paddle. The URDF was made by modifying the sixDOF URDF used in assignment 6.

3 Description of Task

The task was divided into a primary task and secondary task. The primary task is to have the paddle be capable of hitting a ball of any trajectory to any goal point. The secondary task is doing so in a realistic, human-like way which includes having a smooth, realistic trajectory and imitating positions that human tennis players would take.

There are 6 dimensions in the task space (position and orientation of paddle) and 6 dimensions in the joint space (as there are 6 rotary joints). The rotation in the z axis of the paddle is set to complete the task, which led to a 5×6 Jacobian matrix. (To be elaborated in section 4.3 and 4.4).

The task adds the additional freedoms of deciding when in the trajectory the paddle will hit the ball and where on the paddle the ball will hit. The freedom of the trajectory was used to find an optimal arm configuration, which is described in section 4.2. The freedoms of the location on the paddle the ball will hit were not used as we always want the ball to hit in the center of the paddle.

In terms of implementation, the project was divided into 3 subsystems. The subsystem of the ball dealt with the physics of the ball (elaborated in section 5.2) and had the functionality of calculating the position of the ball as needed by the racket (elaborated in section 4.2 and 5.4). The subsystem of the racket deals with finding and moving to the target position and orientation needed to complete the task. There is the overall system of the final project node that deals with the time cycle, spawning and launching a ball, generating a goal, publishing the markers and the arm, and communicating between the ball and the arm (i.e. sending the racket position and orientation to the ball to calculate the collision and sending the position and direction of the ball at a given time

to be used to set the target position and orientation of the paddle).

4 Algorithm and Implementation

4.1 Arm movement

The arm movement is only triggered when a ball is launched. So, the arm initially has a state of WAITINGINIT. As soon as the final_project node launches a ball, it triggers the set target function of the racket. The state of the arm is changed to TOTARGET and the arm moves towards its target position as illustrated in the following subsections. Once it is at the target, the state is changed to WAITINGTARGET and waits for the ball to hit the racket. When the ball hits the racket, the state is again changed to WAITINGINIT and it waits for the next ball to be launched.

4.2 Target position of racket

When a ball is launched, the decision of where along the ball trajectory the racket will hit the ball needs to be made. The optimal position is when the ball is a distance of 0.75 times the arm length from the center of the arm (as elaborated in section 5.4). On the launch of the ball, the position and direction of the ball at the target impact point is calculated by the ball subsystem and used to set the target position of the racket.

To calculate the position at a target distance d, the minimum distance between the ball trajectory and the center of the arm and trajectory of the ball at the minimum distance is first calculated. If p_0 be the center of the arm and p_1 and p_2 be two points along the ball trajectory,

$$
\begin{aligned}
d_{min} &= \frac{|(p_2 - p_1) \times (p_1 - p_0)|}{|p_2 - p_1|} \\
t_d &= -\frac{(p_1 - p_0) \cdot (p_2 - p_1)}{|p_2 - p_1|^2} \\
p_{ball} &= p_1 + t_d(p_2 - p_1) \\
t_{ball} &= \frac{p_{ball} - p_{ball,init}}{v_{ball,init}} \\
v_{ball} &= v_{ball,init} + a_{ball} \cdot t_{ball}
\end{aligned}
$$

If d_{min} is greater or equal to d, the calculated $p_{ball}, t_{ball}, v_{ball}$ is used. If it is smaller, the ball trajectory at d is found by going backwards in the ball trajectory from the point of d_{min} ($\because$ balls are spawned outside of the range of the arm so there must be a point in the trajectory such that $d_{min} == d$). i.e. for a set dt,

```
while (d_ball < d):
```

```
t -= dt
p = p_ball,init + v_ball,init * t + a_ball * t^2 / 2
d_ball = |p - p_0|
```

The paddle always aims to hit the ball at the center of the paddle (i.e. at the origin of the paddle frame). Note that the equations treat the ball trajectory as a line (i.e. doesn't consider the acceleration of the ball). Other approaches are described in section 5.4.

4.3 Target orientation of racket

Given the goal position, the target position of contact, and the ball trajectory at that point (as calculated in section 4.2), the target orientation of the racket is decided. As we are dealing with elastic collision, the normal vector of the surface of the racket (i.e. the z axis of the racket) needs to be the midpoint vector of the vectors between the target position and goal position and the direction of the ball at the point of contact.

$$z = \frac{\vec{\text{direction}}_{ball} + (p_{ball} - goal)}{2}$$

While the z axis of the racket is decided, the x and y axis can be any two perpendicular axes on the surface normal to the z. The arm chooses the x and y axes as follows:

$$\vec{v} = p_{ball} - p_{\text{shoulder of arm}}$$
$$x = \vec{v} \times z$$
$$y = z \times x$$

This is done to ensure that the arm is fully extended towards the target position, as we have noticed choosing any arbitrary axes sometimes constrains the range of the arm. (ex: when the x and y are set such that the wrist is parallel to the world z axis, the length of the wrist is removed from the range of the arm.) Therefore, we want to find the orientation that will extend the racket towards the target position

If there is no goal given to the racket at the point the ball is launched, the racket is oriented such that it sends the ball backwards along its trajectory. (i.e. the z axis of the racket is set to be the direction vector of the ball at the point of contact).

One thing to note is that we are setting the z axis such that the ball only hits one side of the racket, which sometimes causes the racket to do a flip after it hits the ball. To allows the ball to hit the racket on both sides, we would need to let our z axis be either the midpoint vector or the negative of the midpoint vector.

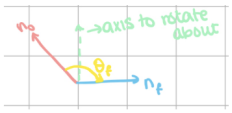
4.4 Inverse Kinematics

Finally, the arm needs to move to the target position and orientation. The trajectory of the arm is generated in the task space. As the final z orientation of the paddle is decided by the ball trajectory and the goal position, the angular velocity along the z axis and the z axis component of the J_w does not need to be considered. Thus, the inverse kinematics is implemented to handle a 5×6 Jacobian matrix as follows:

$$\dot{q} = \begin{bmatrix} J_v \\ R^T J_w \end{bmatrix}^{\dagger} \cdot \begin{bmatrix} v_d + \lambda e_p \\ R^T(w_d + e_n) \end{bmatrix}$$

The z component is dropped from the J_w so the final Jacobian is 5×6. w_d and e_n is calculated as follows:

Let n_0 be the 3rd column of the initial rotation matrix, n_f be the 3rd column of the final (i.e. desired) rotation matrix, and n be the third column of the current rotation matrix at a given point in the trajectory of the paddle. Let $s, \dot{s}$ be the result of the spline (i.e. the returned values of goto(t, duration, 0, 1))



$$\text{axis of rotation } = \frac{n_0 \times n_f}{|n_0 \times n_f|}$$

$$\sin\theta_f = |n_0 \times n_f|$$

$$(\text{or } \theta_f = \cos^{-1}(n_0^T \times n_f) \text{ for } \theta_f > 90^\circ)$$

$$w_d = R^T \cdot \text{ axis } \cdot \theta_f \cdot \dot{s}$$

$$n_d = \text{Rote(axis, } \theta_f \cdot s) \cdot n_0$$

$$e_n = R^T = \vec{n_d} \times \vec{n}$$

A weighted inverse of the Jacobian is used to avoid singularities. Secondary tasks are also added to the final $\dot{q}$, as explained in the next section.

5 Particular Features

5.1 Secondary Tasks

Secondary tasks were used to make sure the robot hits the ball in the optimal configuration. To mimic a human tennis player, the arm has the secondary task of keeping the wrist of the arm straight. This is the natural movement of a human tennis player and makes the robotic arm look more realistic and also helps avoiding the tip of the arm colliding with other joints and links.

To implement the secondary goal, we had defined our q_{goal} as follows:

$$q_{goal} = \begin{bmatrix} 0 \\ -30^\circ \\ 30^\circ \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\dot{q_s} = \lambda * (q_{goal} - q_{last})$$
$$\dot{q_s}[0:3, 0] = 0$$

The first 3 columns of $\dot{q_s}$ is set to 0 as the secondary task is only concerning the wrist.

$$\dot{q}_{extra} = (((I^6 - (J_{winv} \cdot J))) \cdot \dot{q_s})$$
$$\dot{q} = J_{winv} \cdot (V + \lambda \cdot E) + \dot{q}_{extra}$$

where J_{winv} is the weighted inverse calculated using:

$$w = 0.05$$
$$J_{winv} = J^T \cdot (J \cdot J^T + w^2 \cdot I^5)^\dagger$$

Having the secondary task of keeping the elbow slightly bent and in an elbow down position was also considered. However, we decided that keeping the wrist straight is a more important goal as it is very hard to hit the ball if your wrist is tilted badly, but you can still hit the ball if your elbow is straight.

5.2 Ball Physics

The initial position of the ball is randomly generated. The initial velocity of the ball is created by generating a random magnitude and setting the direction to be the vector between the initial position and a random point in the range of the arm. This ensures that the ball has a random trajectory towards an area around the arm. The movement of the ball is simulated by having

$$\vec{v} = \vec{a} \cdot dt$$
$$p = \vec{v} \cdot dt$$

To simulate elastic collision between the ball and the racket, the following process was used:

1. the ball position in world frame is converted to the racket frame.

$$ball_{p,racket} = {R_{racket}}^T \cdot (ball_{p,world} - racket_{p,world})$$

2. The closest point on the racket to the ball is found by clipping the x and y of the $ball_{p,racket}$ to be within the bounds of radius of the racket and the z to be withing the thickness of the racket

3. The closest point in the racket frame is converted back to the world frame.

$$\text{closest point}_{world} = R_{racket} \cdot \text{closest point}_{racket}$$

4. Collision can be checked by checking if

$$|ball_{p,world} - \text{closest point}_{world}| < r_{ball} + thickness_{racket}$$

5. Finally, if collision occurred, the new velocity of the ball is

$$\vec{v} = R_{racket} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix} \cdot {R_{racket}}^T \cdot \vec{v}_{prev}$$

5.3 Arm movement

Initially, the arm movement was implemented such that there was a set initial position and after it hit a ball, it would go to the initial position to wait for the next target. This was done by changing the state of the arm to TOINIT after it hit a ball, generating a trajectory between the previous target position and the initial position using the same inverse kinematics used to go to the target, and changing the state of the arm to WAITINGINIT once it reached the initial position.

This movement simulated the swinging motion of a human tennis player. It was mainly implemented to control the trajectory of the arm to avoid unwanted behavior like the racket going through the links. However, modifications to the inverse kinematics solved the unwanted behavior and the initial position was no longer needed. It ended up adding complications such as hitting the ball while it was moving to the initial position and added unnecessary movement in situations where the previous and next target position is close to each other. Future considerations are setting the initial position in such a way that it doesn't interfere with the ball trajectory, or simulating a swing by changing the trajectory from the previous and next target position to include a point in the middle but further out.

5.4 Optimal position on ball trajectory

The initial approach to deciding where in the ball trajectory the arm would hit it was by simulating a surface by fixing the value of the y or x and calculating when the ball will pass the surface. However, this wasn't the best solution as it limited the range of the ball trajectory (as we would need to choose it such that the ball always passes through the surface) and it caused singularities.

The singularities happened when the arm tried to hit the ball too close or too far from the center of the arm. Thus, we changed our approach such that we will hit the ball when it is as close to a distance of 0.75 times the length of the arm (i.e. essentially finding when the ball passes by a spherical surface of radius 0.75 times the length of the arm around the center of the arm). The ratio of 0.75 was chosen through experimentation, as any ratio above 0.75 or below 0.6 gave Jacobians of very high condition numbers.

The current method of finding the optimal position treats the ball trajectory as a line (i.e. doesn't deal with acceleration of the ball). As the math behind calculating the time that the trajectory passes a spherical surface in 3d is very complicated, an initial attempt was used to have a loop that goes through the ball trajectory to find the optimal point. However, this method took too long to find the optimal point and thus the current method was chosen. A possible method to include the acceleration could be to find a surface that is the wanted distance from the center of the arm and facing the trajectory of the ball (i.e. setting the (x, y) such that the surface is between the arm and the ball) and calculating when and where the ball passes through the surface.

6 Additional Material

6.1 Parameters

The parameters were chosen through experimentation.

- λ for both the primary and secondary task were chosen as 50.
- 0.05 was chosen as the weight of the weighted inverse as a higher weight would cause the arm to miss the target occasionally.

6.2 Progress of implementing inverse kinematics

Implementation of the inverse kinematics was done through a lot of trial and error. All of the implementations used a weighted inverse to deal with singularities.

Initially, the inverse kinematics was as follows:

```
# working kinematics over specfified
t = fmod(t - self.last_time, self.duration)
e = ex() + ey() + ez()
alpha = pi / 2

(s0, s0dot) = goto(t, self.duration, 0.0, 1.0)

pd = p0 + (pf - p0) * s0
vd =      (pf - p0) * s0dot

Rd = r0 @ (s0 * np.linalg.inv(r0)) @ rf
alphadot = alpha * s0dot
wd = alphadot * e

qlast = self.q
(p, R, Jv, Jw) = self.chain.fkin(qlast)
J = np.vstack((Jv, Jw))
V = np.vstack((vd, wd))
E = np.vstack((ep(pd, p), eR(Rd, R)))
```

The inverse kinematics worked as the λ was set to 100, leaving no room for error, and the initial and final position was calculated correctly. However, the rotation axis and the calculation of R_d was incorrect, creating a weird and jumpy trajectory.

To fix this, the following equations were considered:

$$(p, R, J_v, J_w) = \text{fkin}(q_{last})$$

$$\dot{q} = \begin{bmatrix} J_v \\ {}^w|R_p^T J_w \end{bmatrix}^\dagger \cdot \begin{bmatrix} v_d + \lambda e_p \\ {}^w|R_p^T (w_d + \lambda e) \end{bmatrix}$$

Mathematically, this gave a much better $\dot{q}$. However, this was using a 6×6 Jacobian meaning there was only one possible trajectory the racket could take and only one particular position and orientation the racket could be in at a given time. This led to unwanted behavior like the racket going through the links and joints trying to get to the next position. This also meant we could not add any secondary goals and all degrees of freedoms were needed to complete the primary task.

Finally, we arrived at the inverse kinematics described in section 4.4 that uses a 5×6 Jacobian. This gave an additional degree of freedom, allowing the arm to have more than one orientation and position at every given time. This means that the racket is not forced to go through its own links and joints, and can have a more optimal trajectory. It also means we could implement secondary tasks allowing a smoother and more realistic movement.